

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_3512746c-54e\agent-  
a65396fd47f656b0e.jsonl`  
- SHA-256: `a1d363f0449f4ae5431fdf7409760fc2e137d1bb2af24a0358efbb715b9f9ab3`  
- Source modified: `2026-07-06T09:13:39+00:00`  
- Imported at: `2026-07-08T16:00:05+00:00`  
- project: `wf_3512746c-54e`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T09:12:51.503Z)

Reviewing GitHub PR #39 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-e-selective-scan -> main, one commit db87cc0).
It implements ADR 0009 Track E: selective source scanning ? /watch, /unwatch, /channels, /watchlist commands +
an ingestion gate in telethon_client.py that skips unwatched sources. Changed files ONLY:
command_service.py (new /watch etc + _resolve_source), commands.py (re-export patterns), db.py (set_source_watched, list_account_sources), telethon_client.py (the gate),
tests/test_command_service.py (10 tests).
Diff +254/-3. READ docs/adr/0009-selective-channel-scanning.md ? it is the spec this must conform to.

KEY ADR 0009 REQUIREMENTS (quote/verify against the code):

- Line 37-38: "The default is locked as opt-in: DEFAULT 0, so a freshly-onboarded user scans nothing until they opt in."
- Line 44: "...return early if watched is false, before any dedup or filter work."
- Line 130-131: "New channels discovered after the migration default to watched = 0 and must be explicitly watched."
- Line 48-54 (discoverability): lazy-populated rows should be created watched=0 so they appear in /channels for opt-in;
/channels should also upsert candidates from client.get_dialogs().
- Line 64: "/watch <#name|id>" ? resolve by channel name/username OR id.

Ground rules:

- READ actual code (Read/Grep), cite file:line. You MAY run repros with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe (build schema via db._apply_schema on aiosqlite ":memory:"); seed via db.upsert_account + db.upsert_source; drive command_service.CommandService().handle()).
NOTE: real telegram sources have a NUMERIC provider_source_id (str(channel_id) e.g. '-1001234567890'); upsert_channel
creates them with watched=True. The on_new_message gate is inside a '# pragma: no cover' function.
- Authoritative gate ALREADY RUN (don't re-run full suite): ruff check + format clean; pytest 492 passed @ 97.10% (floor 80%).
- Report ONLY defects in THIS PR's diff. Deviation from the ACCEPTED ADR 0009 is a high-value finding. No praise.
Concrete failure scenario per finding, ranked most-severe first, honest severity.
- Severity: blocker (feature broken / data loss on the live path), high (real bug or ADR-contract violation users hit),
medium (edge/latent/half-broken feature), low (hygiene/UX/discoverability).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read the cited code AND docs/adr/0009-selective-channel-scanning.md, trace real control flow, run a repro with

C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe if useful. CONFIRMED only if the defect genuinely ships in THIS PR's diff
and (for ADR-conformance) actually deviates from the accepted ADR. REFUTED if the code conforms / trigger can't occur /
pre-existing. PLAUSIBLE if unsettled. Corrected severity.
FINDING (commands): /channels does not enumerate candidates from client.get_dialogs(), defeating the ADR's discoverability design
severity high | src/tg_video_filter/command_service.py:352
summary: ADR 0009 (lines 48-54, 62) requires /channels to upsert candidate rows from client.get_dialogs() so users can browse and watch channels they have not yet received traffic from; the handler instead only lists already-seen DB rows (db.list_account_sources), and CommandService has no Telegram client at all.
scenario: A freshly-onboarded user (opt-in DEFAULT 0, so scanning nothing ? correct per ADR) runs '/channels' to find something to watch. Because no message has ever been ingested (nothing is watched yet), the sources table is empty and /channels replies '? No sources yet.' The user has no way to discover any of the channels their account is a member of, and therefore no id to pass to /watch. The opt-in default plus lazy population creates a dead end: the ADR explicitly calls the flag default and the get_dialogs enumeration 'orthogonal ? both are required' (line 54), and only one was built.
evidence: _handle_channels (command_service.py:347-361) calls only db.list_account_sources(conn, user_id) and formats the rows; there is no call to client.get_dialogs() and no candidate upsert. CommandService.handle has no client parameter and the class holds no client (command_service.py:181-207; commands.py adapter constructs CommandService() with no client and calls svc.handle without one). get_dialogs is used only in backfill.py:120, never in the command path (grep confirmed). The empty-state text even tells the user to run /backfill instead ? but backfill is a historical-message scan, not the channel enumeration the ADR designed /channels to provide.
rec: Thread the Telethon client (or a provider-agnostic 'list candidate sources' callback) into the /channels path and upsert dialogs as watched=0 candidate rows before listing, per ADR 0009 lines 48-54, so users can browse and opt into channels with no prior traffic. If CommandService must stay client-free, perform the get_dialogs upsert in the Telethon adapter (commands.py) before delegating to handle().

2. assistant (2026-07-06T09:12:57.640Z)

I'll verify this finding adversarially by reading the ADR spec and the actual code.

3. user (2026-07-06T09:12:57.753Z)

```
1      # ADR 0009: Selective channel scanning and multiplexed routing
2
3      - **Status**: Accepted
4      - **Date**: 2026-07-04
5      - **Decision owner**: project owner
6
7      ## Context
8
9      Today the userbot scans every channel/group the account is a member of.
10     `manager.py` registers `events.NewMessage(incoming=True)` with no `chats=`
11     filter, and `on_new_message` processes every video regardless of source. The
12     `channels` table exists but is populated lazily as a record of seen channels;
13     it is not used as a filter.
14
15     The product vision is a multiplexing video re-distribution system:
16
17     - A small set of high-quality source channels (e.g. spiritual, classical
18       music) are scanned for incoming videos.
19     - Matches are routed to themed destination broadcast channels (e.g. a
20       Reggae channel, a Rap channel) that subscribers opt into.
21     - A user therefore subscribes to outputs by topic, not to inputs by source.
22
23     This requires two capabilities the current architecture does not have:
24
25     1. Selective scanning ? choose which source channels are watched, rather
26       than ingesting all of them.
```

```

27 2. **Per-source ? per-destination routing** ? a match from source channel X is
28 delivered to a specific destination channel Y, instead of the single global
29 `delivery_target` (ADR 0005).
30
31 ## Decision
32
33 Add three additive, independently-shippable capabilities:
34
35 ### 1. Selective scanning (watched-flag)
36
37 Add a `watched` flag to the `channels` table. The default is locked as
38 **opt-in: `DEFAULT 0`**, so a freshly-onboarded user scans nothing until they
39 explicitly `/watch` a channel. This matches the multiplexing product shape
40 (sources are a *curated* set) and avoids blindly re-ingesting every chat the
41 account is a member of.
42
43 Gate `on_new_message` at the top: after computing `channel_id`, look up the
44 row and `return` early if `watched` is false, before any dedup or filter work.
45
46 This is a small migration plus ~5 lines in `telethon_client.py`.
47
48 **Discoverability note.** With `DEFAULT 0` plus the current lazy population
49 (a `channels` row is only inserted when a message is first seen), opt-in is
50 necessary but not sufficient for discoverability: a channel the account has
51 never received a message from has no row to flag. The `/channels` command
52 (step 2 below) therefore upserts candidate rows from `client.get_dialogs()`
53 so users can browse and watch channels they have not yet received traffic
54 from. The flag default and the enumeration flow are orthogonal ? both are
55 required.
56
57 ### 2. Channel enumeration and management commands
58
59 New Saved Messages commands (and, via ADR 0008, bot commands) building on the
60 existing `commands.py` pattern:
61
62 - `/channels` ? list the user's channels (via `client.get_dialogs()`), already
63   used in `backfill.py`) with their watched state.
64 - `/watch <#name|id>` ? set `watched=1` for a source channel.
65 - `/unwatch <#name|id>` ? set `watched=0`.
66 - `/watchlist` ? list only watched sources.
67
68 ### 3. Per-source ? per-destination routing rules
69
70 Introduce a `routing_rules` table (the canonical name; issue #25's `routes`
71 refers to the same concept and should be aligned to `routing_rules`), e.g.:
72
73 ```
74 routing_rules (
75     user_id            INTEGER NOT NULL REFERENCES users(id),
76     source_channel_id  INTEGER NOT NULL,
77     dest_channel_id    INTEGER NOT NULL,    -- a broadcast channel / @username
78     filter_config_id   INTEGER,            -- optional per-route filter override
79     PRIMARY KEY (user_id, source_channel_id, dest_channel_id)
80 )
81 ```
82
83 On a match, look up applicable routing rules by `source_channel_id` and
84 deliver to each matching destination. The delivery layer (`delivery.py`)
85 already accepts a `target` per call, so this generalises the existing single
86 `delivery_target` to a per-match lookup. Users with no routing rules fall
87 back to the current global `delivery_target` behaviour (ADR 0005), preserving
88 backwards compatibility.
89
90 ## Consequences
91

```

```

92  **Positive**
93  - Enables the multiplexing product shape: scan curated sources, publish to
94    topical broadcast channels, let subscribers pick outputs by interest.
95  - Each capability is additive and independently deployable; none disturbs the
96    existing FORWARD/LINK/BOTH paths or the metadata-first ingestion (ADR 0003).
97  - Dedup (ADR 0006) still holds: Tier 1 (`document_id`) keys the *original*
98    file, so a routed re-upload is not re-processed.
99  - Channel enumeration reuses `client.get_dialogs()` already present in
100    `backfill.py`; no new Telethon capability is required.
101
102  **Negative**
103  - New schema (`channels.watched`, `routing_rules`) with migrations.
104  - **Routing loop invariant.** With multiplexed routing, the userbot account
105    may be a member of a *destination* broadcast channel and re-see its own
106    output, producing an infinite forward loop. The contract is: **a channel
107    is either a source or a destination, never both.** This is enforced in
108    `on_new_message` by excluding any channel that is a configured destination
109    (`source_channel_id IN (SELECT dest_channel_id FROM routing_rules WHERE
110    user_id = ?)`) before the watched-flag check. Issue #25 Track E implements
111    this; the invariant is stated here so the handler has a contract to code
112    against.
113  - Per-route filter overrides add config complexity; v1 may ship with global
114    filters only and a single implicit route (`source ? global target`), then
115    layer explicit per-route rules later.
116
117  ## Backwards compatibility
118
119  Adding `channels.watched` with `DEFAULT 0` would silently stop every existing
120  install from receiving matches until the user re-watches their channels ? a
121  behaviour change disguised as a schema change. To preserve the current
122  "ingest everything" semantics for existing installs, the migration backfills:
123
124  ```sql
125  ALTER TABLE channels ADD COLUMN watched INTEGER NOT NULL DEFAULT 0;
126  UPDATE channels SET watched = 1 WHERE seen_first_at IS NOT NULL;
127  ```
128
129  i.e. any channel the install has *already* seen traffic from is implicitly
130  watched, so pre-migration users observe no change. New channels discovered
131  after the migration default to `watched = 0` and must be explicitly watched ?
132  which is the intended new UX. This mirrors the "old rows behave exactly as
133  before" pattern established by ADR 0005.
134
135  ## Rollout order
136
137  1. **Selective scanning** (`channels.watched` + `/watch`, `/unwatch`,
138    `/channels`). Smallest unit; immediately useful even without routing.
139  2. **Named destination channels** ? extend `/set delivery_target` to allow
140    switching among a user-defined set, not just one global value.
141  3. **Per-source ? per-destination routing rules** (`routing_rules` table +
142    `/route <source> <dest>` command). The full multiplexing vision.
143
144  Each step is shippable and useful on its own; step 3 is not required to
145  deliver value from steps 1?2.
146
147  ## Alternatives considered
148
149  - **Server-side `chats=` filter at handler registration:** rejected ? the
150    watched set is per-user and mutable at runtime; re-registering Telethon
151    handlers on every `/watch` change is brittle. An in-handler lookup against
152    the `channels` table is simpler and hot-swappable.
153  - **Content-based classification (auto-detect genre):** explicitly out of
154    scope for v1. Routing is user-defined by source channel; automatic topic
155    detection could be a future enhancement but is not required to deliver the
156    multiplexing UX.

```

```

157
158     ## References
159
160     - ADR 0001 (userbot ingestion) ? the capability this selectively constrains.
161     - ADR 0005 (per-user delivery target) ? the single-target delivery this
162       generalises into per-route targets.
163     - ADR 0006 (content dedup) ? still applies; routed re-uploads are keyed by
164       the original `document_id`.
165     - ADR 0008 (bot API onboarding) ? the front-door that exposes `/watch`,
166       `/unwatch`, and routing commands to public users.
167

```

4. user (2026-07-06T09:12:57.901Z)

```

1     """Provider-independent command parsing and execution (ADR 0011, Track D).
2
3     Extracts command logic from the Telethon adapter (`commands.py`) so the
4     same `/set`, `/show` commands work identically through a Bot API
5     front-door. The service is pure: strings in, `CommandResult` out ? no
6     Telethon objects, no network calls.
7
8     See `docs/adr/0007-inline-config-commands.md` for the command model and
9     `docs/adr/0008-bot-api-onboarding-front-door.md` for the Bot API motivation.
10    """
11
12    from __future__ import annotations
13
14    import logging
15    import re
16    from dataclasses import dataclass
17
18    import aiosqlite
19
20    from tg_video_filter import db
21    from tg_video_filter.models import FilterConfig, Source
22
23    log = logging.getLogger(__name__)
24
25    # Key into the shared engine_holder dict (imported by commands.py's adapter).
26    ENGINE_KEY = "engine"
27
28    # --- regex patterns -----
29
30    _SIZE_RE = re.compile(r"^/set\s+size\s+(\d+)\s*([kmgT]?b?)$", re.IGNORECASE)
31    _LENGTH_RE = re.compile(r"^/set\s+(?:max\s+)?length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
32    _MIN_LENGTH_RE = re.compile(r"^/set\s+min\s+length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
33    _MIN_RES_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+(\d+)\s*p?$",
re.IGNORECASE)
34    _SHOW_CONFIG_RE = re.compile(r"^/show\s+config$", re.IGNORECASE)
35    _SHOW_COMMANDS_RE = re.compile(r"^/show\s+commands$", re.IGNORECASE)
36    _EXCLUDE_RE = re.compile(r"^/set\s+exclude\s+(gif|round|nosound)\s+(on|off)$",
re.IGNORECASE)
37    _EXCLUDE_HELP_RE = re.compile(r"^/set\s+exclude\s+", re.IGNORECASE)
38    _MIN_RES_HELP_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+", re.IGNORECASE)
39
40
41    # --- source scanning regex (ADR 0009, Track E) -----
42
43    _WATCH_RE = re.compile(r"^/watch\s+(.+)$", re.IGNORECASE)
44    _UNWATCH_RE = re.compile(r"^/unwatch\s+(.+)$", re.IGNORECASE)
45    _CHANNELS_RE = re.compile(r"^/channels$", re.IGNORECASE)
46    _WATCHLIST_RE = re.compile(r"^/watchlist$", re.IGNORECASE)
47
48    # --- unit parsing -----
49

```

```

50
51 def parse_size_mb(value_str: str, unit: str) -> int:
52     """Parse a size value + unit string into mebibytes."""
53     value = int(value_str)
54     unit = unit.lower().rstrip("b")
55     multipliers: dict[str, int] = {"": 1, "m": 1, "k": 1, "g": 1024, "t": 1024 * 1024}
56     if unit in multipliers:
57         raw = value * multipliers[unit]
58         if unit == "k":
59             raw = max(1, raw // 1024)
60         return raw
61     raise ValueError(f"Unknown size unit: {unit}")
62
63
64 def parse_duration_s(value_str: str, unit: str) -> int:
65     """Parse a duration value + unit string into seconds."""
66     value = int(value_str)
67     unit = unit.lower()
68     multipliers: dict[str, int] = {"": 1, "s": 1, "m": 60, "h": 3600}
69     if unit in multipliers:
70         return value * multipliers[unit]
71     raise ValueError(f"Unknown time unit: {unit}")
72
73
74 # --- formatting helpers -----
75
76
77 def _format_duration(seconds: int | None) -> str:
78     if seconds is None:
79         return "unlimited"
80     if seconds == 0:
81         return "0s"
82     if seconds < 60:
83         return f"{seconds}s"
84     if seconds < 3600:
85         m, s = divmod(seconds, 60)
86         return f"{m}m{s}s" if s else f"{m}m"
87     h, rem = divmod(seconds, 3600)
88     m, s = divmod(rem, 60)
89     parts = f"{h}h"
90     if m:
91         parts += f"{m}m"
92     if s:
93         parts += f"{s}s"
94     return parts
95
96
97 def _format_size(mb: int | None) -> str:
98     if mb is None:
99         return "unlimited"
100     if mb == 0:
101         return "0MB"
102     if mb >= 1024:
103         gb = mb / 1024
104         if gb == int(gb):
105             return f"{int(gb)}GB"
106         return f"{gb:.1f}GB"
107     return f"{mb}MB"
108
109
110 def format_config(cfg: FilterConfig) -> str:
111     """Format a ``FilterConfig`` as a readable message (Telegram markdown)."""
112     lines = [
113         "? **Current Filter Configuration**",
114         "",

```

```

115         f"? **Duration:** {_format_duration(cfg.min_duration_s)} -
{_format_duration(cfg.max_duration_s)}",
116         f"? **Min resolution:** {'?' + str(cfg.target_height) + 'p' if
cfg.target_height else 'disabled'}",
117         f"? **Max file size:** {_format_size(cfg.max_size_mb)}",
118         f"? **Delivery mode:** `{cfg.delivery_mode.value}`",
119         f"? **Delivery target:** `{cfg.delivery_target}`",
120         "",
121         "? **Content exclusions:**",
122         f"    GIF/animated: {'?' excluded' if cfg.exclude_gif else '?' allowed'}",
123         f"    Round videos: {'?' excluded' if cfg.exclude_round else '?' allowed'}",
124         f"    Silent (no audio): {'?' excluded' if cfg.exclude_nosound else '?' allowed'}",
125     ]
126     return "\n".join(lines)
127
128
129 def format_commands() -> str:
130     """Return a help message listing all available commands."""
131     return "\n".join(
132         [
133             "? **Available Commands**",
134             "",
135             "? **Filter config**",
136             "`/show config` ? Show current filter settings",
137             "`/set size <N><unit>` ? Set max file size (e.g. `/set size 10MB`)",
138             "    Units: `MB` (default), `GB`, `KB` ? case-insensitive, optional trailing
139             `B`",
140             "`/set length <N><unit>` ? Set max video duration (e.g. `/set length 15m`)",
141             "    or `/set max length <N><unit>` ? same as above",
142             "    Units: `s` (default), `m`, `h` ? case-insensitive",
143             "`/set min length <N><unit>` ? Set min video duration (e.g. `/set min length
144             5s`)",
145             "    Units: `s` (default), `m`, `h` ? case-insensitive",
146             "`/set min resolution <N>p` ? Set min video height (e.g. `/set min
147             resolution 720p`)",
148             "    or `/set min res <N>p` ? same as above",
149             "",
150             "? **Content exclusions** (all off by default)",
151             "`/set exclude gif on|off` ? Exclude Telegram GIFs/animated documents",
152             "`/set exclude round on|off` ? Exclude circular video messages",
153             "`/set exclude nosound on|off` ? Exclude videos without audio\n"
154             "    Note: GIFs are usually silent; `exclude nosound` will catch most GIFs
155             too.",
156             "",
157             "? **Backfill**",
158             "`/backfill [N][hdw]` ? Scan last N hours/days/weeks of channel history
159             (default 90d)",
160             "`/stop` ? Cancel running backfill",
161             "",
162             "\u2139\u2013 **Help**",
163             "`/show commands` ? Show this help message",
164         ]
165     )
166
167 # --- command result -----
168
169 @dataclass
170 class CommandResult:
171     """The output of executing a command ? provider-independent.
172
173     The adapter (Telethon or Bot API) uses ``reply_text`` to send the response
174     and ``config_changed`` to decide whether to hot-reload the filter engine.
175     """

```

```

173
174     reply_text: str = ""
175     config_changed: bool = False
176
177
178 # --- command service -----
179
180
181 class CommandService:
182     """Provider-independent command execution.
183
184     Parses a command string against an account's current config and DB,
185     returning a ``CommandResult``. No Telethon objects involved ? pure string
186     in, effects out. The Telethon adapter (``commands.py``) handles the
187     transport (checking message.out, sending the reply); the Bot API adapter
188     (future) calls ``handle()`` with the ``from.id`` from the update.
189
190     Usage::
191
192         svc = CommandService()
193         result = await svc.handle(user_id, text, conn, current_cfg)
194         if result.config_changed:
195             engine_holder["engine"] = FilterEngine(current_cfg)
196         await send_reply(result.reply_text)
197     """
198
199 # -- public API -----
200
201 async def handle(
202     self,
203     user_id: int,
204     text: str,
205     conn: aiosqlite.Connection,
206     current_cfg: FilterConfig,
207 ) -> CommandResult:
208     """Parse and execute a command text, returning the reply + change flag."""
209     text = text.rstrip()
210
211     # --- /set size <value><unit> -----
212     m = _SIZE_RE.match(text)
213     if m:
214         value_str, unit = m.group(1), m.group(2) or ""
215         # NOTE: ValueError is unreachable here ? the regex already validates
216         # the unit against [kmgT]?b?, so parse_size_mb can never raise.
217         # Kept for defensive parity with the original commands.py refactor.
218         try:
219             new_mb = parse_size_mb(value_str, unit)
220         except ValueError:
221             return CommandResult("? Invalid size format. Use e.g. `/set size
10MB`.")
222         if new_mb < 1:
223             return CommandResult("? Size must be at least 1 MB (`FilterConfig`
constraint).")
224         cfg = await self._update(conn, user_id, current_cfg, max_size_mb=new_mb)
225         return CommandResult(
226             f"? Max file size set to
**[_format_size(new_mb)]**.\n\n{format_config(cfg)}",
227             config_changed=True,
228         )
229
230     # --- /set length <value><unit> -----
231     m = _LENGTH_RE.match(text)
232     if m:
233         value_str, unit = m.group(1), m.group(2) or ""
234         try:

```



```

235         new_secs = parse_duration_s(value_str, unit)
236     except ValueError:
237         return CommandResult("? Invalid duration format. Use e.g. `/set length
15m`.")
238
239     min_secs = current_cfg.min_duration_s or 0
240     if new_secs < min_secs:
241         return CommandResult(
242             f"? Max duration ({_format_duration(new_secs)}) must be "
243             f">= min duration ({_format_duration(min_secs)}).")
244
245     cfg = await self._update(conn, user_id, current_cfg,
max_duration_s=new_secs)
246     return CommandResult(
247         f"? Max duration set to
**[_format_duration(new_secs)]**.\n\n{format_config(cfg)}",
248         config_changed=True,
249     )
250
251     # --- /set min length <value><unit> -----
252     m = _MIN_LENGTH_RE.match(text)
253     if m:
254         value_str, unit = m.group(1), m.group(2) or ""
255         try:
256             new_min = parse_duration_s(value_str, unit)
257         except ValueError:
258             return CommandResult("? Invalid duration format. Use e.g. `/set min
length 5s`.")
259
260     max_secs = current_cfg.max_duration_s or 0
261     if max_secs > 0 and new_min > max_secs:
262         return CommandResult(
263             f"? Min duration ({_format_duration(new_min)}) must be "
264             f"<= max duration ({_format_duration(max_secs)}).")
265
266     cfg = await self._update(conn, user_id, current_cfg, min_duration_s=new_min)
267     return CommandResult(
268         f"? Min duration set to
**[_format_duration(new_min)]**.\n\n{format_config(cfg)}",
269         config_changed=True,
270     )
271
272     # --- /set min resolution <N>p -----
273     m = _MIN_RES_RE.match(text)
274     if m:
275         new_height = int(m.group(1))
276         if new_height < 1:
277             return CommandResult("? Resolution must be at least 1p.")
278         cfg = await self._update(conn, user_id, current_cfg,
target_height=new_height)
279     return CommandResult(
280         f"? Min resolution set to **{new_height}p**.\n\n{format_config(cfg)}",
281         config_changed=True,
282     )
283
284     if _MIN_RES_HELP_RE.match(text):
285         return CommandResult(
286             "? Usage: `/set min resolution <N>p` (e.g. `/set min resolution 720p`)"
287         )
288
289     # --- /set exclude <gif|round|nosound> <on|off> -----
290     m = _EXCLUDE_RE.match(text)
291     if m:
292         what = m.group(1).lower()
293         state = m.group(2).lower()
294         enabled = state == "on"
295         labels = {"gif": "GIF/animated", "round": "round videos", "nosound": "silent

```

```

videos"}
294         field_map = {
295             "gif": "exclude_gif",
296             "round": "exclude_round",
297             "nosound": "exclude_nosound",
298         }
299         cfg = await self._update(conn, user_id, current_cfg, **{field_map[what]:
enabled})
300         return CommandResult(
301             f"? {labels[what]} exclusion: **{'ON' if enabled else 'OFF'}**."
302             f"\n\n{format_config(cfg)}",
303             config_changed=True,
304         )
305
306     if _EXCLUDE_HELP_RE.match(text):
307         return CommandResult(
308             "? Usage: `/set exclude <gif|round|nosound> <on|off>`\n"
309             "Example: `/set exclude gif on`"
310         )
311
312     # --- /show config -----
313     if _SHOW_CONFIG_RE.match(text):
314         return CommandResult(format_config(current_cfg))
315
316     # --- source scanning commands (ADR 0009, Track E) -----
317
318     m = _WATCH_RE.match(text)
319     if m:
320         return await self._handle_watch(conn, user_id, m.group(1).strip(),
current_cfg)
321
322     m = _UNWATCH_RE.match(text)
323     if m:
324         return await self._handle_unwatch(conn, user_id, m.group(1).strip(),
current_cfg)
325
326     if _CHANNELS_RE.match(text):
327         return await self._handle_channels(conn, user_id)
328
329     if _WATCHLIST_RE.match(text):
330         return await self._handle_watchlist(conn, user_id)
331
332     # --- /show commands -----
333     if _SHOW_COMMANDS_RE.match(text):
334         return CommandResult(format_commands())
335
336     # Not a recognised command ? return empty result (noise).
337     return CommandResult()
338
339     # -- source scanning handlers (ADR 0009, Track E) -----
340
341     async def _handle_watch(
342         self, conn: aiosqlite.Connection, user_id: int, target: str, _cfg: FilterConfig
343     ) -> CommandResult:
344         """Parse `/watch <username|channel_id>` and toggle `watched=1`."""
345         src = await self._resolve_source(conn, user_id, target)
346         if src is None:
347             return CommandResult(
348                 f"? Source `{target}` not found. Use `/channels` to list "
349                 f"available sources.\n\nSources are discovered as messages "
350                 f"arrive; a channel you haven't received traffic from may not "
351                 f"appear yet."
352             )
353         await db.set_source_watched(conn, user_id, src.id, True) # type: ignore[arg-
type]

```

```

354         label = src.title or src.provider_source_id
355         return CommandResult(f"? Now watching **{label}**
({src.provider_source_id}).")
356
357     async def _handle_unwatch(
358         self, conn: aiomysqlite.Connection, user_id: int, target: str, _cfg: FilterConfig
359     ) -> CommandResult:
360         """Parse `/unwatch <@username|channel_id>` and toggle `watched=0`."""
361         src = await self._resolve_source(conn, user_id, target)
362         if src is None:
363             return CommandResult(
364                 f"? Source `{target}` not found. Use `/channels` to list sources."
365             )
366         await db.set_source_watched(conn, user_id, src.id, False) # type: ignore[arg-
type]
367         label = src.title or src.provider_source_id
368         return CommandResult(f"? Stopped watching **{label}**
({src.provider_source_id}).")
369
370     async def _handle_channels(self, conn: aiomysqlite.Connection, user_id: int) ->
CommandResult:
371         """List all known sources with watch status."""
372         sources = await db.list_account_sources(conn, user_id)
373         if not sources:
374             return CommandResult(
375                 "? **No sources yet.**\n\n"
376                 "Sources are discovered when a new message arrives from a "
377                 "channel. Run `/backfill` to discover historical channels, "
378                 "or wait for new traffic."
379             )
380         lines = ["? **All sources**", ""]
381         for s in sources:
382             icon = "?" if s.watched else "?"
383             label = s.title or s.provider_source_id
384             lines.append(f"{icon} `{s.provider_source_id}` ? {label}")
385         return CommandResult("\n".join(lines))
386
387     async def _handle_watchlist(self, conn: aiomysqlite.Connection, user_id: int) ->
CommandResult:
388         """List only watched sources."""
389         sources = await db.list_account_sources(conn, user_id, watched_only=True)
390         if not sources:
391             return CommandResult(
392                 "? **No watched sources.**\n\n"
393                 "Use `/channels` to see available sources, then `/watch <id>` "
394                 "to start scanning."
395             )
396         lines = ["? **Watched sources**", ""]
397         for s in sources:
398             label = s.title or s.provider_source_id
399             lines.append(f"? `{s.provider_source_id}` ? {label}")
400         return CommandResult("\n".join(lines))
401
402     async def _resolve_source(
403         self, conn: aiomysqlite.Connection, user_id: int, target: str
404     ) -> Source | None:
405         """Resolve a user-provided source identifier to a `Source` row.
406
407         Accepts `@username` or a numeric channel id string. The `@` prefix
408         is stripped before lookup since Telegram channel ids are numeric.
409
410         target = target.lstrip("@")
411         # Try by provider-native id first (the common case).
412         src = await db.get_source_by_provider_id(conn, user_id, "telegram", target)
413         if src is not None:

```

```

414         return src
415         # Fall back: try with @ prefix in the target column (just in case).
416         return await db.get_source_by_provider_id(conn, user_id, "telegram",
f"@{target}")
417
418     # -- internal -----
419
420     async def _update(
421         self,
422         conn: aiosqlite.Connection,
423         user_id: int,
424         current: FilterConfig,
425         max_size_mb: int | None = None,
426         max_duration_s: int | None = None,
427         min_duration_s: int | None = None,
428         target_height: int | None = None,
429         exclude_gif: bool | None = None,
430         exclude_round: bool | None = None,
431         exclude_nosound: bool | None = None,
432     ) -> FilterConfig:
433         """Apply partial updates to the account's config, persist, and return."""
434         updates: dict[str, object] = {}
435         if max_size_mb is not None:
436             updates["max_size_mb"] = max_size_mb
437         if max_duration_s is not None:
438             updates["max_duration_s"] = max_duration_s
439         if min_duration_s is not None:
440             updates["min_duration_s"] = min_duration_s
441         if target_height is not None:
442             updates["target_height"] = target_height
443         if exclude_gif is not None:
444             updates["exclude_gif"] = exclude_gif
445         if exclude_round is not None:
446             updates["exclude_round"] = exclude_round
447         if exclude_nosound is not None:
448             updates["exclude_nosound"] = exclude_nosound
449         cfg = current.model_copy(update=updates) if updates else current
450         await db.save_filter_config(conn, user_id, cfg)
451         log.info(
452             "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
453             "target_height=%s exclude_gif=%s exclude_round=%s exclude_nosound=%s",
454             user_id,
455             cfg.max_size_mb,
456             cfg.min_duration_s,
457             cfg.max_duration_s,
458             cfg.target_height,
459             cfg.exclude_gif,
460             cfg.exclude_round,
461             cfg.exclude_nosound,
462         )
463         return cfg
464
465     __all__ = [
466         "ENGINE_KEY",
467         "CommandResult",
468         "CommandService",
469         "_format_duration",
470         "_format_size",
471         "format_config",
472         "format_commands",
473         "parse_duration_s",
474         "parse_size_mb",
475     ]
476
477

```

5. assistant (2026-07-06T09:13:02.739Z)

Now let me look at db.py functions and check whether there's any client threading in the command path via commands.py.

6. user (2026-07-06T09:13:04.360Z)

```
src\tg_video_filter\backfill.py:120:    dialogs = await client.get_dialogs()
```

7. user (2026-07-06T09:13:04.722Z)

```
1      """Config commands ? Telethon adapter over the provider-independent `CommandService`.
2
3      This module is a thin Telethon adapter. It handles Telethon-specific transport
4      (checking `message.out`, `peer_id`, sending replies) and delegates command
5      parsing and config mutation to `command_service.CommandService` (ADR 0011, Track D).
6
7      Re-exports regex patterns and unit-parsers for backwards-compatibility with
8      existing tests (`tests/test_commands.py`).
9
10     See `docs/adr/0007-inline-config-commands.md` for the command model and
11     `docs/adr/0008-bot-api-onboarding-front-door.md` for the Bot API front-door.
12     """
13
14     from __future__ import annotations
15
16     import logging
17     from collections.abc import Callable
18
19     import aiosqlite
20     from telethon import TelegramClient
21     from telethon.errors import RPCError
22
23     from tg_video_filter import db
24     from tg_video_filter.command_service import ENGINE_KEY, CommandResult, CommandService
25     from tg_video_filter.filters import FilterEngine
26     from tg_video_filter.models import FilterConfig, User
27
28     log = logging.getLogger(__name__)
29
30     from tg_video_filter.command_service import ( # noqa: E402, F401
31         _CHANNELS_RE,
32         _EXCLUDE_HELP_RE,
33         _EXCLUDE_RE,
34         _LENGTH_RE,
35         _MIN_LENGTH_RE,
36         _MIN_RES_HELP_RE,
37         _MIN_RES_RE,
38         _SHOW_COMMANDS_RE,
39         _SHOW_CONFIG_RE,
40         _SIZE_RE,
41         _UNWATCH_RE,
42         _WATCH_RE,
43         _WATCHLIST_RE,
44         _format_duration,
45         _format_size,
46         format_commands,
47         format_config,
48         parse_duration_s,
49         parse_size_mb,
50     )
51
52
53     def make_config_command_handler(
54         user: User,
```

```

55         conn: aiosqlite.Connection,
56         engine_holder: dict[str, FilterEngine],
57     ) -> Callable:
58         svc = CommandService()
59
60         async def on_command(event: object) -> None:
61             message = getattr(event, "message", None)
62             if message is None:
63                 return
64             if not getattr(message, "out", False):
65                 return
66             peer = getattr(message, "peer_id", None)
67             peer_user_id = getattr(peer, "user_id", None) if peer else None
68             if peer_user_id != user.id:
69                 return
70
71             text: str = getattr(message, "message", "") or ""
72             client: TelegramClient = getattr(event, "client", None) # type:
ignore[assignment]
73
74             # Defer DB load ? skip non-command self-messages (review finding #2).
75             if not text.startswith("/"):
76                 return
77
78             current_cfg = await db.load_filter_config(conn, user.id)
79             result: CommandResult = await svc.handle(user.id, text, conn, current_cfg)
80
81             if result.config_changed:
82                 new_cfg = await db.load_filter_config(conn, user.id)
83                 engine_holder[ENGINE_KEY] = FilterEngine(new_cfg)
84
85             if result.reply_text:
86                 await _reply(client, message, result.reply_text)
87
88         return on_command
89
90
91 # --- backwards-compat helpers (test_commands.py references) -----
92
93
94 async def _update_config(
95     conn: aiosqlite.Connection,
96     user_id: int,
97     max_size_mb: int | None = None,
98     max_duration_s: int | None = None,
99     min_duration_s: int | None = None,
100     target_height: int | None = None,
101     exclude_gif: bool | None = None,
102     exclude_round: bool | None = None,
103     exclude_nosound: bool | None = None,
104 ) -> FilterConfig:
105     """Deprecated wrapper ? delegates to ``CommandService._update``.
106
107     Kept for backwards-compatibility with ``tests/test_commands.py`` which
108     imports ``_update_config`` from here. New code should call
109     :meth:`CommandService.handle` which returns a :class:`CommandResult`.
110     """
111     svc = CommandService()
112     cfg = await db.load_filter_config(conn, user_id)
113     return await svc._update(
114         conn,
115         user_id,
116         cfg,
117         max_size_mb=max_size_mb,
118         max_duration_s=max_duration_s,

```

```

119         min_duration_s=min_duration_s,
120         target_height=target_height,
121         exclude_gif=exclude_gif,
122         exclude_round=exclude_round,
123         exclude_nosound=exclude_nosound,
124     )
125
126
127     async def _reply(client: TelegramClient, to_message: object, text: str) -> None:
128         try:
129             await client.send_message("me", text)
130         except (OSError, TimeoutError, RPCError):
131             log.warning("config_reply_failed", exc_info=True)
132
133
134     __all__ = [
135         "ENGINE_KEY",
136         "_EXCLUDE_RE",
137         "_LENGTH_RE",
138         "_MIN_LENGTH_RE",
139         "_MIN_RES_RE",
140         "_SHOW_CONFIG_RE",
141         "_SHOW_COMMANDS_RE",
142         "_EXCLUDE_HELP_RE",
143         "_MIN_RES_HELP_RE",
144         "_SIZE_RE",
145         "_format_duration",
146         "_format_size",
147         "_update_config",
148         "format_commands",
149         "format_config",
150         "make_config_command_handler",
151         "parse_duration_s",
152         "parse_size_mb",
153     ]
154

```

8. assistant (2026-07-06T09:13:14.648Z)

The finding is confirmed by the code. Let me verify the key claims:

1. `\_handle\_channels` (command_service.py:370-385) calls only `db.list\_account\_sources` - confirmed, no `get\_dialogs`, no candidate upsert.
2. `get\_dialogs` is used only in `backfill.py:120` - confirmed by grep.
3. `CommandService.handle` has no `client` parameter (lines 201-207) - confirmed.
4. The Telethon adapter `commands.py` has a `client` available (line 72) but does NOT pass it to `svc.handle` (line 79) and does no `get\_dialogs` upsert - confirmed.

Now let me verify the ADR truly makes this a mandatory requirement and not just a "note." The ADR is clearly authoritative on this:

- Line 51-54: "The `/channels` command (step 2 below) **therefore** upserts candidate rows from `client.get\_dialogs()`" so users can browse and watch channels they have not yet received traffic from. The flag default and the enumeration flow are orthogonal ? **both** are required".

- Line 62: "`/channels` ? list the user's channels (**via** `client.get\_dialogs()`", already used in `backfill.py`) with their watched state."

The ADR is unambiguous. `/channels` is specified to enumerate via `get\_dialogs()`, and the ADR explicitly names discoverability as a required, orthogonal capability. The implementation only lists already-seen DB rows.

Let me check the scenario's dead-end claim: with DEFAULT 0 and lazy population, a fresh user has an empty sources table.

9. user (2026-07-06T09:13:16.264Z)

```

99:         watched          INTEGER NOT NULL DEFAULT 0,
522:     # 1a. Backfill sources from channels first (known title + watched=1). Order
523:     #     matters: channels-first so title/watched data wins; the videos-based
528:         (account_id, provider, provider_source_id, title, kind, watched,
539:     #     ? the channels-based row above survives with its title + watched=1.
543:         (account_id, provider, provider_source_id, kind, watched, seen_first_at)
851:         watched=bool(row["watched"]),
1105:     """Insert a source row, or update title/watched on conflict.
1112:         (account_id, provider, provider_source_id, title, kind, watched,
1117:         watched=excluded.watched,
1125:         int(source.watched),
1387:async def upsert_channel(conn:aiosqlite.Connection, channel:Channel) -> Channel:
1402:         watched=True, # back-compat: pre-ADR-0011 channels were all watched
1537:async def set_source_watched(
1538:     conn:aiosqlite.Connection, account_id:int, source_id:int, watched:bool
1540:     """Toggle the `watched` flag on a source (ADR 0009, Track E)."""
1542:     "UPDATE sources SET watched = ? WHERE id = ? AND account_id = ?",
1543:     (int(watched), source_id, account_id),
1548:async def list_account_sources(
1549:     conn:aiosqlite.Connection, account_id:int, *, watched_only:bool = False
1551:     """List all sources for an account, optionally filtering to watched only."""
1555:     if watched_only:
1556:         query += " AND watched = 1"
1557:     query += " ORDER BY watched DESC, provider_source_id ASC"
1578:         watched=True,
1746:     "set_source_watched",

```

10. assistant (2026-07-06T09:13:27.869Z)

The finding holds up under adversarial scrutiny. ``list_account_sources`` (db.py:1548) purely reads existing ``sources`` rows; there's no ``get_dialogs`` enumeration anywhere in the command path. The DEFAULT 0 (db.py:99) plus lazy population means a fresh user's sources table is empty until a message from a watched channel arrives ? but nothing is watched, so no ingestion happens, so nothing ever populates. That is the dead-end the scenario describes.

Now the one honest question on severity: the scenario's "dead end" isn't quite absolute. The empty-state text points users to ``/backfill``, which does call ``get_dialogs()`` and can upsert source rows. So a workaround exists (run ``/backfill`` to populate rows, then ``/channels`` shows them). That mitigates the "no way to discover" claim somewhat ? it's not a total dead end, and users retain access via a documented-in-UI alternative. But ``/backfill`` scans historical message `*content*` (an expensive operation), not a lightweight channel enumeration, and it only surfaces channels that have historical videos matching ? it is not the browse-all-dialogs discovery the ADR designed. The ADR-specified capability is genuinely absent.

This is a real deviation from the accepted ADR (an explicit "both are required" mandate), and it ships in this PR's diff (``_handle_channels`` is new code). It degrades but does not fully break the feature ? a partial workaround exists via ``/backfill``. That places it at high (ADR-contract violation users hit), consistent with the reported severity, though the existence of the ``/backfill`` mitigation keeps it from being a blocker.

11. user (2026-07-06T09:13:39.003Z)

Structured output provided successfully